EXP 5

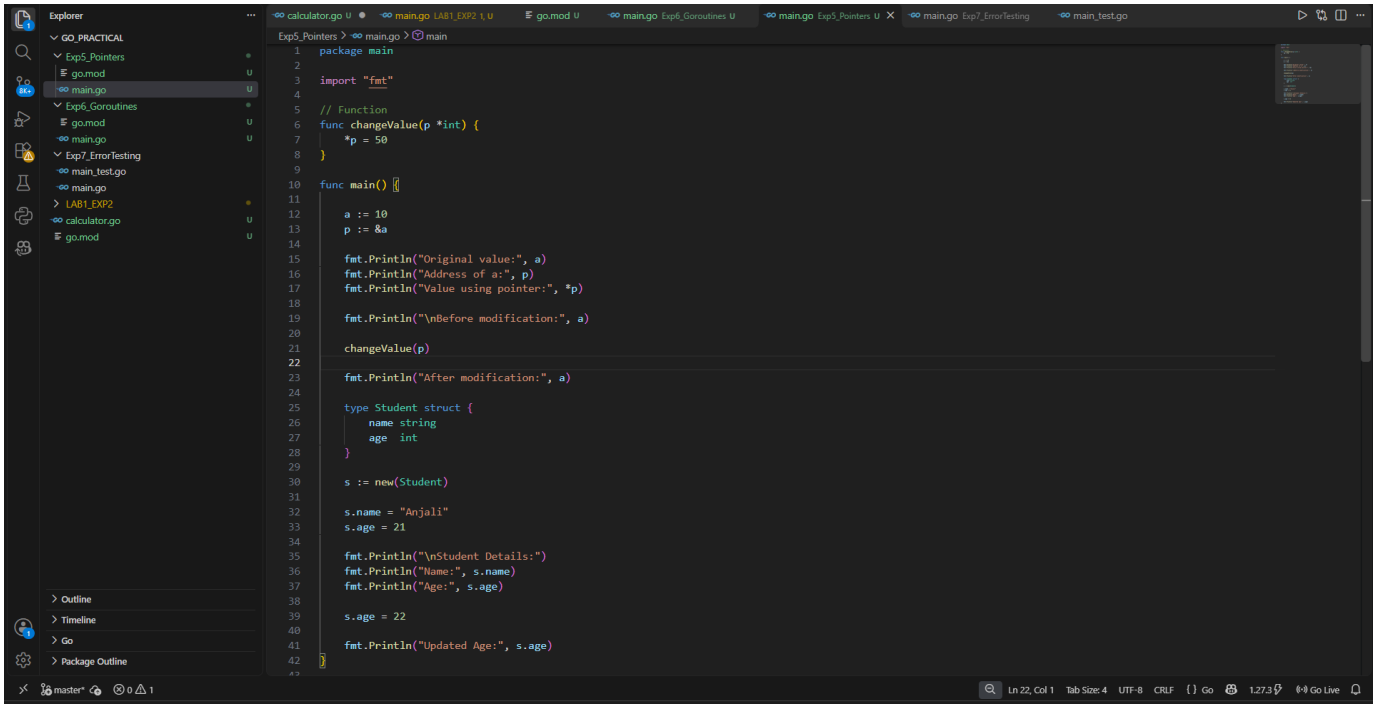
1. Pointer Manipulation — Referencing and Dereferencing

Write a Go program demonstrating pointer usage:

(1) declare a variable, print its address using & and access its value using *;

(2) write a function that accepts a pointer parameter and modifies the original variable (pass-by-reference) — print before and after values;

(3) allocate a struct using new() and access/modify its fields through the pointer.



```go
package main

import "fmt"

// Function
func changeValue(p *int) {
    *p = 50
}

func main() {

    a := 10
    p := &a

    fmt.Println("Original value:", a)
    fmt.Println("Address of a:", p)
    fmt.Println("Value using pointer:", *p)

    fmt.Println("\nBefore modification:", a)

    changeValue(p)

    fmt.Println("After modification:", a)

    type Student struct {
        name string
        age  int
    }

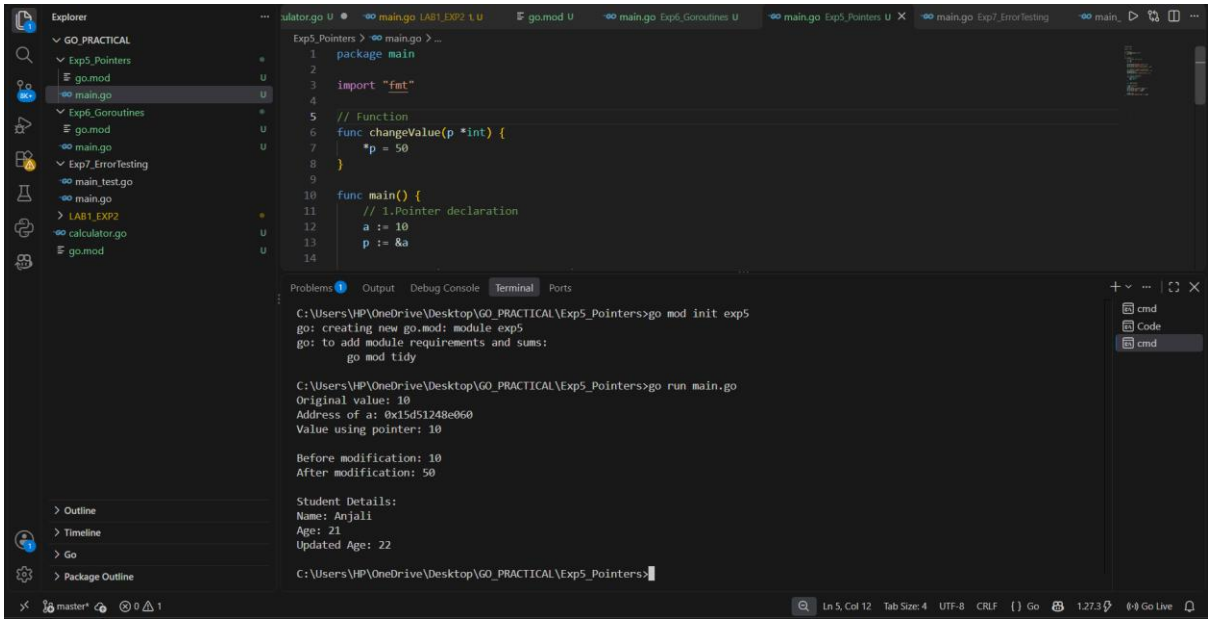
    s := new(Student)

    s.name = "Anjali"
    s.age = 21

    fmt.Println("\nStudent Details:")
    fmt.Println("Name:", s.name)
    fmt.Println("Age:", s.age)

    s.age = 22

    fmt.Println("Updated Age:", s.age)
}
```



```
C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp5_Pointers>go mod init exp5
go: creating new go.mod: module exp5
go: to add module requirements and sums:
        go mod tidy

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp5_Pointers>go run main.go
Original value: 10
Address of a: 0x15d51248e060
Value using pointer: 10

Before modification: 10
After modification: 50

Student Details:
Name: Anjali
Age: 21
Updated Age: 22

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp5_Pointers>
```